Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)

Lab Report NO: 02
Course Title: AI Lab
Course Code: CSE 316
Section: 232-D5

Lab Experiment Name: Implementation of Depth-First Search and Breadth-First Search Traversal

Student Details

	Name	ID
1.	Ashab Uddin	232002274

Lab Date : **22/02/2026**
Submission Date : **28/02/2026**
Course Teacher's Name : **Babe Sultana**

Lab Report Status

Marks: …… **Signature: .....**
Comments: ..... **Date: .....**

1. INTRODUCTION

This lab report is based on the implementation of Depth-First Search (DFS**)** and Breadth-First Search (BFS), which are two fundamental graph traversal algorithms used to visit all nodes of a graph or tree.

DFS explores one branch deeply before backtracking. It is implemented using recursion or a stack and is useful for path finding and cycle detection.

BFS explores nodes level by level. It uses a queue and is commonly used to find the shortest path in an unweighted graph.

The main difference between them is that DFS goes deep first, while BFS explores level by level.

2. OBJECTIVES/AIM

- To understand how to represent states and nodes in a graph.
- To understand how Bread-First Search (BFS) works on a two-dimensional (2D) plane.
- To understand how Depth-First Search (DFS) works on a two-dimensional (2D) plane.

3. PROCEDURE

1. Procedure for BFS Traversal on a 2D Plane

1. Take user input for grid size N.
2. Generate an N × N matrix.
3. Randomly place obstacles (0 = obstacle, 1 = free cell).
4. Print the generated matrix.
5. Take user input for starting state and goal state.
6. Initialize a queue for BFS and a visited set.
7. Add the starting node to the queue.
8. Traverse the grid level by level using BFS rules (Up, Down, Left, Right).
9. Stop when the goal state is reached or queue becomes empty.

2. Procedure to Print Required Moves During BFS Traversal

1. During BFS traversal, track the current node.
2. For each movement (Up, Down, Left, Right), determine the direction.
3. When moving to a valid adjacent tile, print:
   Example: Moving Down -> (2, 3)
4. Continue printing each move until the goal is found.

3. Procedure for Robot Traversal Using BFS and Printing Path

1. Take grid size and generate matrix with random obstacles.
2. Take start and destination input.
3. Use BFS with a queue.
4. Maintain a parent dictionary to store the previous node of each visited node.
5. When destination is found, stop BFS.
6. Use a recursive function to trace back from destination to start using parent information.
7. Print the final path from start to destination.

4. Procedure for DFS Traversal on a 2D Plane

1. Take user input for grid size N.
2. Generate an N × N matrix with random obstacles.
3. Print the matrix.
4. Take starting and goal states as input.
5. Initialize a visited set.
6. Use recursion (or stack) to implement DFS.
7. Explore deeply in one direction before backtracking.
8. Stop when goal is reached or all possible paths are explored.

5. Procedure to Find Path from Source to Destination Using DFS

1. Take grid size and generate matrix with obstacles.
2. Take source and destination input.
3. Use recursive DFS function.
4. Maintain a parent dictionary to store traversal history.
5. When destination is found, return True.
6. Use recursive path printing function to print path from source to destination using parent information.
7. If destination is not found, print "No Path Found".

4. IMPLEMENTATION:

Code:

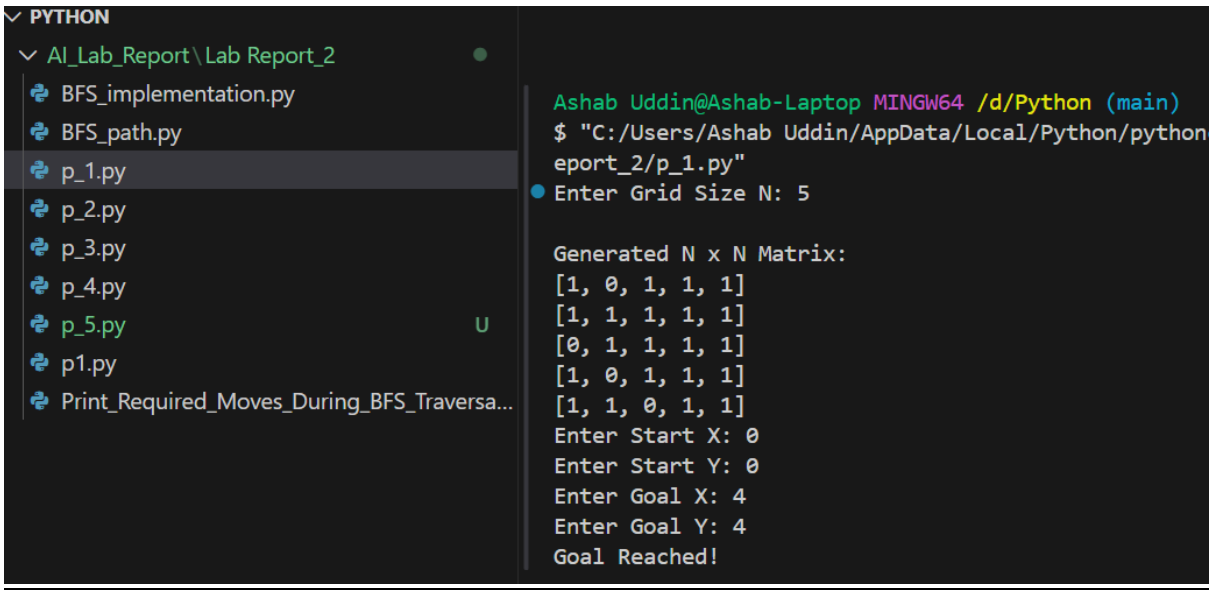
<u>Problem1:</u>

```
import random
from collections import deque

# Create grid with random obstacles
def create_grid(n):
    grid = []
    for i in range(n):
        row = []
        for j in range(n):
```

```python
            row.append(random.choice([1, 1, 1, 0]))  # More free cells (1)
        grid.append(row)
    return grid

# Print Matrix
def print_grid(grid):
    print("\nGenerated N x N Matrix:")
    for row in grid:
        print(row)

# BFS traversal
def bfs(grid, start, goal):
    n = len(grid)
    queue = deque([start])
    visited = set([start])

    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    while queue:
        x, y = queue.popleft()

        if (x, y) == goal:
            print("Goal Reached!")
            return True

        for dx, dy in directions:
            nx, ny = x+dx, y+dy

            if 0<=nx<n and 0<=ny<n:
                # Check FREE cell (1)
                if grid[nx][ny] == 1 and (nx,ny) not in visited:
                    queue.append((nx,ny))
                    visited.add((nx,ny))

    print("No Path Found")
    return False


# MAIN
n = int(input("Enter Grid Size N: "))
grid = create_grid(n)
print_grid(grid)

sx = int(input("Enter Start X: "))
sy = int(input("Enter Start Y: "))
gx = int(input("Enter Goal X: "))
gy = int(input("Enter Goal Y: "))

start = (sx, sy)
```

```
goal = (gx, gy)

# Ensure start and goal are free (1)
grid[sx][sy] = 1
grid[gx][gy] = 1

bfs(grid, start, goal)
```

Output Problem-1:



Problem2:

```
import random
from collections import deque

def create_grid(n):
    # More free cells (1), fewer obstacles (0)
    return [[random.choice([1,1,1,0]) for _ in range(n)] for _ in range(n)]

def print_grid(grid):
    print("\nGenerated N x N Matrix:")
    for row in grid:
        print(row)

def bfs_with_moves(grid, start, goal):
    n = len(grid)
    queue = deque([start])
    visited = set([start])

    directions = [(-1,0,"Up"),
                  (1,0,"Down"),
                  (0,-1,"Left"),
```

```
                    (0,1,"Right")]

    while queue:
        x, y = queue.popleft()

        if (x,y) == goal:
            print("\nGoal Reached!")
            return

        for dx, dy, move in directions:
            nx, ny = x+dx, y+dy

            if 0<=nx<n and 0<=ny<n:
                # Now check FREE cell (1)
                if grid[nx][ny] == 1 and (nx,ny) not in visited:
                    print(f"Moving {move} -> ({nx}, {ny})")
                    queue.append((nx,ny))
                    visited.add((nx,ny))

    print("No Path Found")


# MAIN
n = int(input("Enter Grid Size N: "))
grid = create_grid(n)
print_grid(grid)

sx = int(input("Enter Start X: "))
sy = int(input("Enter Start Y: "))
gx = int(input("Enter Goal X: "))
gy = int(input("Enter Goal Y: "))

start = (sx, sy)
goal = (gx, gy)

# Ensure start and goal are free (1)
grid[sx][sy] = 1
grid[gx][gy] = 1

bfs_with_moves(grid, start, goal)
```

Output Problem-2:

```
PYTHON
AI_Lab_Report\Lab Report_2
BFS_implementation.py
BFS_path.py
p_1.py
p_2.py
p_3.py
p_4.py
p_5.py                                U
p1.py
Print_Required_Moves_During_BFS_Traversa...

Ashab Uddin@Ashab-Laptop MINGW64 /d/Python (main)
$ "C:/Users/Ashab Uddin/AppData/Local/Python/pythoncore-3
Enter Grid Size N: 6

Generated N x N Matrix:
[1, 1, 1, 1, 0, 0]
[0, 1, 1, 1, 1, 1]
[0, 0, 1, 0, 1, 1]
[1, 1, 0, [redacted], 1, 0]
[0, 1, 0, 0, 0, 0]
[1, 1, 0, 0, 0, 0]
Enter Start X: 0
Enter Start Y: 0
Enter Goal X: 3
Enter Goal Y: 3
Moving Right -> (0, 1)
Moving Down -> (1, 1)
Moving Right -> (0, 2)
Moving Right -> (1, 2)
Moving Right -> (0, 3)
Moving Down -> (2, 2)
Moving Right -> (1, 3)
Moving Down -> (3, 2)
Moving Right -> (1, 4)
Moving Left -> (3, 1)
Moving Right -> (3, 3)
Moving Down -> (2, 4)
Moving Right -> (1, 5)
Moving Down -> (4, 1)
Moving Left -> (3, 0)

Goal Reached!
```

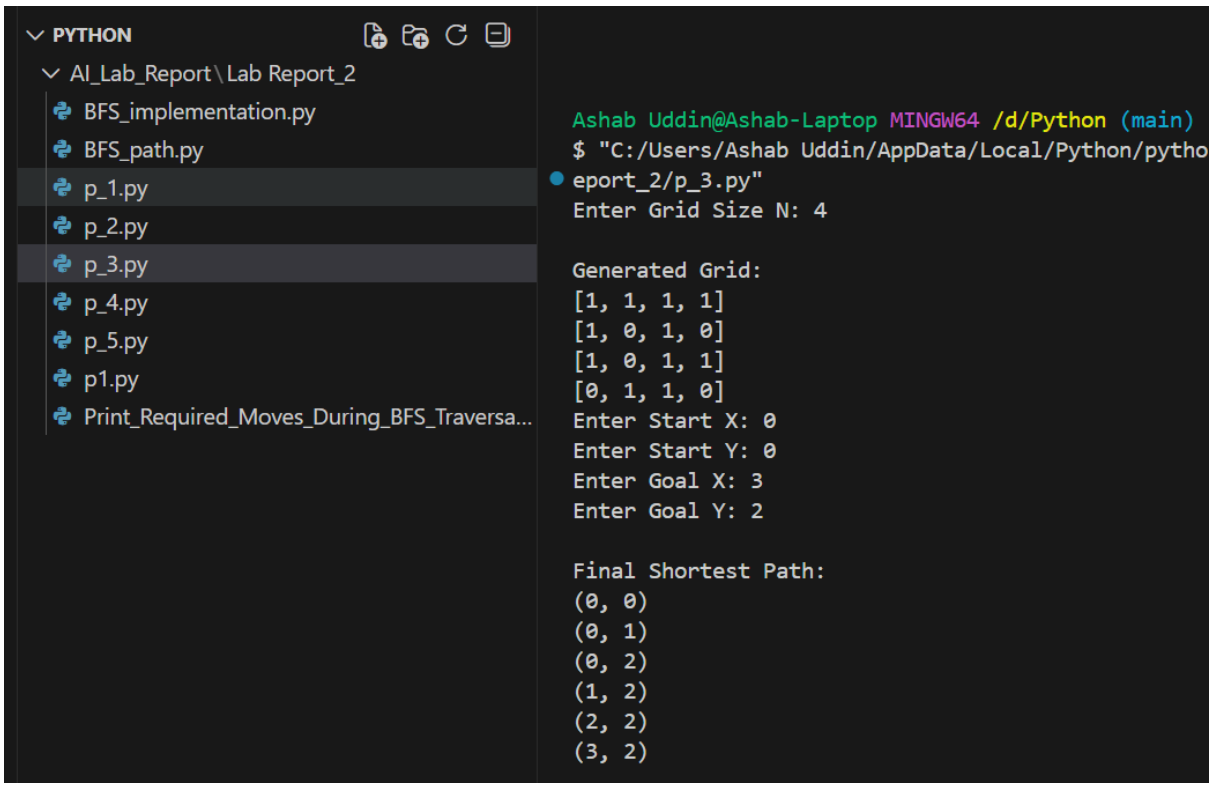
Problem3:

```python
import random
from collections import deque

def create_grid(n):
    # More free cells (1), fewer obstacles (0)
    return [[random.choice([1,1,1,0]) for _ in range(n)] for _ in range(n)]

def print_grid(grid):
    print("\nGenerated Grid:")
    for row in grid:
        print(row)

def bfs_with_parent(grid, start, goal):
    n = len(grid)
    queue = deque([start])
    visited = set([start])
    parent = {}
```

```python
    directions = [(-1,0),(1,0),(0,-1),(0,1)]

    while queue:
        x, y = queue.popleft()

        if (x,y) == goal:
            return parent

        for dx, dy in directions:
            nx, ny = x+dx, y+dy

            if 0<=nx<n and 0<=ny<n:
                # Check FREE cell (1)
                if grid[nx][ny] == 1 and (nx,ny) not in visited:
                    queue.append((nx,ny))
                    visited.add((nx,ny))
                    parent[(nx,ny)] = (x,y)

    return None


# Recursive path printing
def print_path(parent, start, goal):
    if parent is None:
        print("No Path Found")
        return

    if goal == start:
        print(start)
        return

    print_path(parent, start, parent[goal])
    print(goal)


# MAIN
n = int(input("Enter Grid Size N: "))
grid = create_grid(n)
print_grid(grid)

sx = int(input("Enter Start X: "))
sy = int(input("Enter Start Y: "))
gx = int(input("Enter Goal X: "))
gy = int(input("Enter Goal Y: "))

start = (sx, sy)
goal = (gx, gy)

# Ensure start and goal are free (1)
```

```
grid[sx][sy] = 1
grid[gx][gy] = 1

parent = bfs_with_parent(grid, start, goal)

print("\nFinal Shortest Path:")
print_path(parent, start, goal)
```

Output Problem -3:



Problem4:

```
import random

# Create grid with random obstacles
def create_grid(n):
    grid = []
    for i in range(n):
        row = []
        for j in range(n):
            row.append(random.choice([1, 1, 1, 0]))  # More free cells (1)
        grid.append(row)
    return grid


# Print grid
def print_grid(grid):
```

```python
    print("\nGenerated Grid:")
    for row in grid:
        print(row)


# DFS traversal
def dfs(grid, current, goal, visited):
    n = len(grid)
    x, y = current

    if current == goal:
        print("Goal Reached!")
        return True

    visited.add(current)

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < n and 0 <= ny < n:
            # Check FREE cell (1) instead of 0
            if grid[nx][ny] == 1 and (nx, ny) not in visited:
                if dfs(grid, (nx, ny), goal, visited):
                    return True

    return False


# MAIN
n = int(input("Enter Grid Size N: "))
grid = create_grid(n)
print_grid(grid)

sx = int(input("Enter Start X: "))
sy = int(input("Enter Start Y: "))
gx = int(input("Enter Goal X: "))
gy = int(input("Enter Goal Y: "))

start = (sx, sy)
goal = (gx, gy)

# Ensure start and goal are free (1)
grid[sx][sy] = 1
grid[gx][gy] = 1

visited = set()
```

```
if not dfs(grid, start, goal, visited):
    print("No Path Found")
```

Output Problem-4:

```
PYTHON
AI_Lab_Report\Lab Report_2
BFS_implementation.py
BFS_path.py
p_1.py
p_2.py
p_3.py
p_4.py
p_5.py
p1.py
Print_Required_Moves_During_BFS_Traversa...

Ashab Uddin@Ashab-Laptop MINGW64 /d/Python (main)
$ "C:/Users/Ashab Uddin/AppData/Local/Python/pytho
eport_2/p_4.py"
[1, 1, 0, 1, 1]
[1, 1, 1, 0, 1]
[1, 1, 1, 1, 1]
[1, 1, 0, 1, 1]
Enter Start X: 1
Enter Start Y: 1
Enter Goal X: 3
Enter Goal Y: 4
Goal Reached!
```

Problem5:

```
import random

# Create grid with random obstacles
def create_grid(n):
    return [[random.choice([1,1,1,0]) for _ in range(n)] for _ in range(n)]

# Print grid
def print_grid(grid):
    print("\nGenerated Grid:")
    for row in grid:
        print(row)

# DFS with parent tracking (without movement printing)
def dfs(grid, current, goal, visited, parent):
    n = len(grid)
    x, y = current

    if current == goal:
        return True

    visited.add(current)

    directions = [(-1,0),
                  (1,0),
                  (0,-1),
                  (0,1)]

    for dx, dy in directions:
```

```python
        nx, ny = x + dx, y + dy

        if 0 <= nx < n and 0 <= ny < n:
            if grid[nx][ny] == 1 and (nx, ny) not in visited:
                parent[(nx, ny)] = (x, y)

                if dfs(grid, (nx, ny), goal, visited, parent):
                    return True

    return False


# Recursive function to print path
def print_path(parent, start, goal):
    if goal == start:
        print(start)
        return

    print_path(parent, start, parent[goal])
    print(goal)


# MAIN
n = int(input("Enter Grid Size N: "))
grid = create_grid(n)
print_grid(grid)

sx = int(input("Enter Start X: "))
sy = int(input("Enter Start Y: "))
gx = int(input("Enter Goal X: "))
gy = int(input("Enter Goal Y: "))

start = (sx, sy)
goal = (gx, gy)

# Ensure start and goal are free
grid[sx][sy] = 1
grid[gx][gy] = 1

visited = set()
parent = {}

found = dfs(grid, start, goal, visited, parent)

if found:
    print("\nGoal Reached!")
    print("\nPath from Source to Destination:")
    print_path(parent, start, goal)
else:
```

```
    print("No Path Found")
```

Output Problem-5:

```
PYTHON
AI_Lab_Report\Lab Report_2
BFS_implementation.py
BFS_path.py
p_1.py
p_2.py
p_3.py
p_4.py
p_5.py                                  M
p1.py
Print_Required_Moves_During_BFS_Traversa...
```

```
(4, 5)

Ashab Uddin@Ashab-Laptop MINGW64 /d/Python (main)
$ "C:/Users/Ashab Uddin/AppData/Local/Python/pythonc
Enter Grid Size N: 6

Generated Grid:
[0, 1, 0, 1, 1, 1]
[1, 1, 1, 1, 0, 1]
[1, 0, 0, 1, 0, 1]
[1, 1, 1, 0, 1, 1]
[0, 1, 0, 1, 0, 1]
[1, 0, 1, 0, 1, 1]
Enter Start X: 0
Enter Start Y: 0
Enter Goal X: 3
Enter Goal Y: 4

Goal Reached!

Path from Source to Destination:
(0, 0)
(1, 0)
(1, 1)
(1, 2)
(1, 3)
(0, 3)
(0, 4)
(0, 5)
(1, 5)
(2, 5)
(3, 5)
(3, 4)
```

6. ANALYSIS AND DISCUSSION

In this lab, we implemented Breadth-First Search (BFS) and Depth-First Search (DFS) traversal algorithms on a 2D plane. BFS explores nodes level by level and guarantees the shortest path in an unweighted grid. DFS explores deeply before backtracking and is useful for path existence checking.

Both algorithms successfully demonstrate different traversal strategies and help understand graph search techniques in practical scenarios such as robot path finding.